Learning Objectives

Learners will be able to…

- **Distinguish between max-heaps and min-heaps by identifying their structural properties and understanding their roles in various applications.**
- **Illustrate the structure of complete binary trees and explain how heaps are represented in memory, particularly using array-based implementations.**
- **Demonstrate the process of inserting elements into a heap, including the steps of maintaining heap properties and analyzing the time complexity of the insertion operation.**
- **Execute the deletion operation in heaps, detailing the steps involved in removing elements while preserving heap structure and evaluating the efficiency of the operation.**
- **Identify and explain various real-world applications of heaps, emphasizing their utility in scenarios like priority queue implementation and efficient data management.**

What Is a Heap?

A heap is a specialized tree-based data structure that satisfies the heap property. In a **max-heap**, for any given node `i` other than the root, the value of `i` is less than or equal to the value of its parent. Conversely, in a **min-heap**, the value of `i` is greater than or equal to the value of its parent. This property makes heaps useful in a variety of applications, most notably in implementing *priority queues*.

Max-Heaps and Min-Heaps

Max-Heap

In a max-heap, the *largest* element is at the root, and this property is recursively true for all sub-trees in the heap.

Example of a max-heap:

```
      42
     /  \
   29    18
  / \   / \
14 19 11 10
```

Min-Heap

Conversely, in a min-heap, the *smallest* element is at the root, and this property holds for all sub-trees.

Example of a min-heap:

```
      10
     /  \
   14    11
  / \   / \
19 29 18 42
```

Heap Representation

Heaps are usually represented as binary trees, but they are typically implemented using arrays for efficiency. In the array representation, if the root element is at index `0`, then for any given element at index `i`:

- The left child is at index `2 * i + 1`

- The right child is at index `2 * i + 2`
- The parent is at index `(i - 1) / 2` (floored)

This representation is **space-efficient** and makes it easy to traverse the heap.

Heap Operations

The primary operations associated with heaps are:

- **Insertion**: Adding a new element to the heap while maintaining the heap property. This involves placing the element at the end of the heap (in the array representation) and then "bubbling up" or "heapifying up" the element to its correct position.
- **Deletion**: This usually refers to removing the root element from the heap, which is the maximum in a max-heap or the minimum in a min-heap. After removing the root, the last element in the heap is placed at the root, and then the heap is "heapified down" to maintain the heap property.
- **Heapify**: This operation transforms a binary tree into a heap. It's essential when creating a heap from an unsorted array and during the deletion operation to restructure the heap.

Applications of Heaps

Heaps are particularly useful in several areas:

- **Priority Queues**: Heaps are ideal for implementing priority queues where elements are processed based on their priority rather than their order in the queue.
- **Heap Sort**: This sorting algorithm uses a heap to sort elements. It repeatedly removes the root element (the highest or lowest, depending on the heap type) and restructures the heap until all elements are sorted.
- **Graph Algorithms**: In algorithms like Dijkstra's and Prim's, heaps (often implemented as priority queues) are used for efficiently finding the next node to process.

Conclusion

Heaps offer an efficient way to manage data that needs to be constantly ordered or prioritized. Their implementation using arrays makes them space-efficient, and their operations are relatively straightforward yet powerful. Understanding heaps is a stepping stone to mastering more complex data structures and algorithms, especially those involving priority-based data processing and efficient sorting methods.

In the next lessons, we will explore the specific operations of heaps in more detail, including how to implement them and their applications in various algorithms.

Implementing a Heap

In this lesson, we will begin implementing a heap. We will initially focus on the basic structure and later delve into specific heap operations.

Setting Up the Basic Structure

We start by defining the heap class. Unlike a tree-based structure, heaps are typically implemented using arrays for efficiency. The class has already been started for you, just add the constructor below to your code.

```
    // Constructor initializes the heap with a given capacity
public:
    HeapImplementation(int capacity) {
        heap = new int[capacity];
        this->capacity = capacity;
        size = 0;
    }
```

Add this class to your student file.

Inserting Elements and Resizing the Heap

Next, let's implement a basic method for inserting elements into the heap. This will place a new element at the end of the heap. Note that when a heap is created, it has a fixed size. Thus, at some point, it will become full. To allow for the continuation of adding elements, we will also include a `resize` method to double the size of the heap so that we can keep inserting elements. We won't focus on maintaining the heap property just yet though.

```
public:
    void resize() {
        // Resizes the heap to double its current size when full
        int* resizedHeap = new int[capacity * 2];
        for (int i = 0; i < capacity; i++) {
            resizedHeap[i] = heap[i];
        }
        delete[] heap;
        heap = resizedHeap;
        capacity *= 2;
    }

public:
    void insert(int value) {
        // Inserts a new value into the heap, resizing if
        necessary
        if (size == capacity) {
            resize();
        }
        heap[size] = value;
        size++;
    }
```

Incorporate these methods into your student file.

important

IMPORTANT

You'll notice that there are comments regarding additional methods within the code file such as `heapifyUp`, `delete`, etc. Please leave these sections empty for now. As you progress through the assignment, you will fill in these sections with code as instructed.

Testing the Heap

In order to see the contents of our heap, we'll want to include a `printHeap` method. Additionally, let's create a simple `main` method to test our code.

```
public:
    void printHeap() {
        // Prints the current elements in the heap
        for (int i = 0; i < size; i++) {
            std::cout << heap[i] << " ";
        }
        std::cout << std::endl;
    }
```

```
    int main() {
        // Main method to test the functionality of the
        // HeapImplementation class
        HeapImplementation heap(10);

        heap.insert(15);
        heap.insert(10);
        heap.insert(20);

        heap.printHeap();

        return 0;
    }
```

After adding these methods to your file, you'll be able to test your code.

Student's Code File

Your code file should look something like this:

```
class HeapImplementation {
private:
    int* heap;
    int size;
    int capacity;

public:
    HeapImplementation(int capacity) {
        heap = new int[capacity];
        this->capacity = capacity;
        size = 0;
    }

    void resize() {
        int* resizedHeap = new int[capacity * 2];
        for (int i = 0; i < capacity; i++) {
            resizedHeap[i] = heap[i];
        }
        delete[] heap;
        heap = resizedHeap;
        capacity *= 2;
    }

    void insert(int value) {
        if (size == capacity) {
            resize();
        }
        heap[size] = value;
        size++;
    }

    //heapifyUp and helper methods
```

```
    //delete method

    //heapifyDown and helper methods

    //search method

    void printHeap() {
        for (int i = 0; i < size; i++) {
            std::cout << heap[i] << " ";
        }
        std::cout << std::endl;
    }
};

int main() {
    HeapImplementation heap(10);

    heap.insert(15);
    heap.insert(10);
    heap.insert(20);

    heap.printHeap();

    return 0;
}
```

Complete Binary Trees and Heap Representation

A complete binary tree is a key concept in understanding heaps. In a complete binary tree, every level, except possibly the last, is completely filled, and all nodes are as far left as possible. This structure is crucial for the efficient operation of heaps.

Example of a Complete Binary Tree:

```
      15
     /  \
   10    12
  / \   /
 25 30 36
```

In this example, every level of the tree is fully filled except for the last level, which is filled from left to right.

Heap as a Complete Binary Tree

Heaps are a specific type of complete binary tree. They maintain the heap property (max-heap or min-heap) while also ensuring that the tree is complete. This completeness is vital for the efficient performance of heap operations, as it allows the heap to be efficiently represented as an array.

Array Representation of Heaps

The array representation of a heap leverages the properties of a complete binary tree. Given a heap element's index in the array, it's straightforward to calculate the indices of its parent, left child, and right child.

- **Parent**: The parent of the node at index `i` is at index `(i - 1) / 2`.
- **Left Child**: The left child of the node at index `i` is at index `2 * i + 1`.
- **Right Child**: The right child of the node at index `i` is at index `2 * i + 2`.

Example of Heap Array Representation:

For the complete binary tree shown earlier, the array representation would be:

```
{ 15, 10, 12, 25, 30, 36 }
```

Advantages of Array Representation

1. **Space Efficiency**: The array representation of a heap does not require pointers to child nodes, making it more space-efficient than a typical tree representation.
2. **Easy Access**: The parent-child relationship is easily computed, allowing for quick access to nodes during heap operations.
3. **Simplicity**: The array-based representation simplifies the implementation of heap operations like insertion and deletion.

Conclusion

Understanding the concept of complete binary trees and their array representation is fundamental to grasping how heaps function. This knowledge is crucial for implementing heap operations efficiently and for understanding the underlying mechanics of heap-based algorithms like heap sort and priority queues. In the next lessons, we will delve deeper into specific heap operations and their applications.

Heap Operations - Insertion

Inserting a new element into a heap is a fundamental operation that maintains the heap's structure and properties. The process involves two main steps: adding the new element to the end of the heap and then "heapifying" to restore the heap property.

Step 1: Adding the New Element

The new element is initially added to the end of the heap, which is the next available position in the array representation. This ensures that the tree remains complete.

Example: Inserting `20` into a Max-Heap

Initial Max-Heap (Array Representation):

```
{ 42, 29, 18, 14, 19, 11, 10 }
```

After adding `20`:

```
{ 42, 29, 18, 14, 19, 11, 10, 20 }
```

Step 2: Heapify Up

After adding the new element, the heap may not satisfy the heap property. To restore this property, we perform a "heapify up" process. This involves comparing the added element with its parent and swapping them if the heap property is violated. This process is repeated until the heap property is restored.

Heapify Up Process:

1. Compare the new element with its parent.
2. If the heap property is violated (e.g., the new element is larger than its parent in a max-heap), swap the element with its parent.
3. Repeat the process until the new element is in the correct position, or it becomes the root.

Example of Heapify Up:

Continuing with the previous example, `20` is compared with its parent `11`. Since `20` is greater, they are swapped.

```
      42
     /  \
   29    18
  / \   / \
 14 19 20 10
   /
  11
```

The process continues until the heap property is restored.

Complexity of Insertion

The time complexity of inserting an element into a heap is $O(logn)$, where n is the number of elements in the heap. This is because the height of a complete binary tree is $logn$, and the heapify up process may traverse from the leaf to the root in the worst case.

Conclusion

Heap insertion is a critical operation that enables the dynamic addition of elements while maintaining the heap's structural and property integrity. Understanding this process is essential for implementing efficient priority queues and for the heap sort algorithm. In the next lesson, we will explore the deletion operation in heaps and its impact on the heap structure.

Initial Max-Heap (Array Representation):

```
{ 42, 29, 18, 14, 19, 11, 10 }
```

After adding `20`:

```
 { 42, 29, 18, 14, 19, 11, 10, 20 }
```

In the tree representation, `20` is added as the left child of `14`.

```
      42
     /  \
   29    18
  / \   / \
 14 19 11 10
/
20
```

Heapify Up Process:

1. Compare `20` with its parent `14`.
2. Since `20` is greater than `14` (in a max-heap), they are swapped.

After the swap, the heap looks like this:

```
       42
      /  \
    29    18
   / \   / \
  20 19 11 10
 /
14
```

3. Now, compare `20` with its new parent `29`. Since `20` is less than `29`, no further swaps are needed, and the heap property is maintained.

The final heap after insertion and heapify up is:

```
       42
      /  \
    29    18
   / \   / \
  20 19 11 10
 /
14
```

Implementing Insertion

In this lesson, we'll refine the insertion method from our `HeapImplementation` class earlier. Not only will we be able to add new elements to the end of the heap, but we'll also be able to perform "heapify up" to maintain the heap structure.

Step 1: Adding the New Element

First, we modify the insert method to include a call to `heapifyUp`, which will be defined subsequently. Remember, we already have a resizing mechanism from the previous lesson.

```
void insert(int value) {
    // Modified to include heapify up operation
    if (size == capacity) {
        resize();
    }

    heap[size] = value;
    heapifyUp(); // This method will help maintain heap
        structure
    size++;
}
```

Step 2: Heapify Up

Now, we implement the "heapify up" process. This method ensures the heap property is maintained after each insertion.

```
void heapifyUp() {
    // Restores the heap property by moving the newly
    // added element to its correct position
    int index = size;
    while (hasParent(index) && parent(index) < heap[index]) {
        swap(getParentIndex(index), index);
        index = getParentIndex(index);
    }
}
```

Additional helper methods such as `hasParent`, `parent`, `getParentIndex`, `swap`, and `printHeap` are also provided and should be included in the code file as well.

```
bool hasParent(int i) { return getParentIndex(i) >= 0; }
int parent(int i) { return heap[getParentIndex(i)]; }
int getParentIndex(int i) { return (i - 1) / 2; }

void swap(int i, int j) {
    int temp = heap[i];
    heap[i] = heap[j];
    heap[j] = temp;
}

void printHeap() {
    for (int i = 0; i < size; i++) {
        std::cout << heap[i] << " ";
    }
    std::cout << std::endl;
}
```

These helper methods are crucial for the heapify up process, allowing us to compare and swap an element with its parent when necessary. Once again, please leave the remaining methods within the code file empty for now.

Testing the Implementation

Let's test our enhanced insertion operation with a new `main` method. Create a heap with a capacity of `10`. Then use the `insert` method to add eight items. Finally, print the heap.

```
int main() {
    HeapImplementation heap(10);

    heap.insert(42);
    heap.insert(29);
    heap.insert(18);
    heap.insert(14);
    heap.insert(19);
    heap.insert(11);
    heap.insert(10);
    heap.insert(20);

    heap.printHeap();

    return 0;
}
```

Revise your `main` method so that it looks like above. It will help you test the insertion operation.

Student's Code File

The complete `HeapImplementation` class should now look like this:

```cpp
class HeapImplementation {
private:
    int* heap;
    int size;
    int capacity;

public:
    HeapImplementation(int capacity) {
        heap = new int[capacity];
        this->capacity = capacity;
        size = 0;
    }

    void resize() {
        int* resizedHeap = new int[capacity * 2];
        for (int i = 0; i < capacity; i++) {
            resizedHeap[i] = heap[i];
        }
        delete[] heap;
        heap = resizedHeap;
        capacity *= 2;
    }

    void insert(int value) {
        if (size == capacity) {
            resize();
        }
        heap[size] = value;
        heapifyUp();
        size++;
    }

    void heapifyUp() {
        int index = size;
        while (hasParent(index) && parent(index) < heap[index])
        {
            swap(getParentIndex(index), index);
            index = getParentIndex(index);
        }
    }

    bool hasParent(int i) { return getParentIndex(i) >= 0; }
    int parent(int i) { return heap[getParentIndex(i)]; }
    int getParentIndex(int i) { return (i - 1) / 2; }

    void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    void printHeap() {
        for (int i = 0; i < size; i++) {
            std::cout << heap[i] << " ";
        }
        std::cout << std::endl;
    }
```

```
    //delete method

    //heapifyDown and helper methods

    //search method

};

int main() {
    HeapImplementation heap(10);

    heap.insert(42);
    heap.insert(29);
    heap.insert(18);
    heap.insert(14);
    heap.insert(19);
    heap.insert(11);
    heap.insert(10);
    heap.insert(20);

    heap.printHeap();

    return 0;
}
```

The code should produce `42 29 18 20 19 11 10 14` as the output due to the implementation of `heapifyUp` which maintains the max-heap structure. This structure was maintained up through the addition of the element `20`. `20` is larger than its parent `14` causing a swap to occur. After the swap, the structure is once again maintained.

Heap Operations - Deletion

Deletion in a heap typically refers to the removal of the root element. In a max-heap, this is the largest element, and in a min-heap, it's the smallest. This operation is crucial in many applications, such as implementing priority queues, where the most (or least) important item is frequently removed.

Deletion Process

The deletion process in a heap involves several steps to maintain the heap property:

1. **Remove the Root**: The root element is removed from the heap.
2. **Replace with Last Element**: The last element in the heap (the rightmost, bottom element) is moved to the root position.
3. **Heapify Down**: The new root may violate the heap property, so it's necessary to perform a "heapify down" operation. This process involves comparing the node with its children and swapping it with one of them (the larger one in a max-heap or the smaller one in a min-heap) if necessary. This step is repeated until the heap property is restored.

Example of Deletion in a Max-Heap

Consider the following max-heap:

```
      42
     /  \
   29    18
  / \   / \
 14 19 11 10
```

Step 1: Remove the Root

We remove `42`, the root of the heap.

Step 2: Replace with Last Element

The last element `10` is moved to the root.

```
      10
     /  \
   29    18
  / \   /
14 19 11
```

Step 3: Heapify Down

`10` is compared with its children `29` and `18`. Since `29` is the larger child in this max-heap, `10` is swapped with `29`.

```
      29
     /  \
   10    18
  / \   /
14 19 11
```

Now, `10` is compared with its children `14` and `19`. It's swapped with the larger child `19`.

```
      29
     /  \
   19    18
  / \   /
14 10 11
```

The heap property is now restored.

Complexity of Deletion

The time complexity of deletion in a heap is $O(logn)$, where n is the number of elements in the heap. This is due to the heapify down process, which, in the worst case, traverses from the root to the leaf along the height of the tree, which is $logn$.

Conclusion

Deletion in a heap is a fundamental operation that allows for the efficient removal of the highest (or lowest) priority element. The process involves removing the root, replacing it with the last element, and then heapifying down to maintain the heap property. This operation is essential for applications like priority queues and heap sort, where elements are frequently removed in order of priority.

Implementing Deletion

In this lesson, we'll implement the deletion operation for a heap, focusing on removing the root element and maintaining the heap's structure.

Deletion Process

The deletion process involves two main steps:

1. **Remove the Root**: Replace the root of the heap with the last element.
2. **Heapify Down**: Restore the heap property by heapifying down from the root.

Let's implement these steps in our `HeapImplementation` class.

Implementing Root Removal

First, we modify the heap to remove the root and replace it with the last element.

```
int deleteRoot() {
    // Removes the root element and replaces it with the last
    // element, then performs heapify down
    if (size == 0) {
        throw std::logic_error("Heap is empty");
    }

    int root = heap[0];
    heap[0] = heap[size - 1];
    size--;
    heapifyDown();
    return root;
}
```

Add this `deleteRoot` method to your C++ file in the specified section.

Implementing Heapify Down

Now, let's implement the "heapify down" process.

```
void heapifyDown() {
    // Adjusts the heap after deletion to maintain the
    // heap property by moving elements downwards
    int index = 0;
    while (hasLeftChild(index)) {
        int largerChildIndex = getLeftChildIndex(index);
        if (hasRightChild(index) && rightChild(index) >
        leftChild(index)) {
            largerChildIndex = getRightChildIndex(index);
        }

        if (heap[index] < heap[largerChildIndex]) {
            swap(index, largerChildIndex);
        } else {
            break;
        }

        index = largerChildIndex;
    }
}

bool hasLeftChild(int i) { return getLeftChildIndex(i) < size; }
bool hasRightChild(int i) { return getRightChildIndex(i) < size;
        }
int leftChild(int i) { return heap[getLeftChildIndex(i)]; }
int rightChild(int i) { return heap[getRightChildIndex(i)]; }
int getLeftChildIndex(int i) { return 2 * i + 1; }
int getRightChildIndex(int i) { return 2 * i + 2; }
```

Include the additional methods in your C++ file as well to facilitate the heapify down process.

Note that there will be one remaining method, `search`, which will be implemented later. For now, please continue to leave it blank.

Testing the Deletion

Let's test our deletion operation with a revised `main` method.

```
int main() {
    HeapImplementation heap(10);
    heap.insert(42);
    heap.insert(29);
    heap.insert(18);
    heap.insert(14);
    heap.insert(19);
    heap.insert(11);
    heap.insert(10);

    std::cout << "Heap before deletion:" << std::endl;
    heap.printHeap();

    heap.deleteRoot();

    std::cout << "Heap after deletion:" << std::endl;
    heap.printHeap();

    return 0;
}
```

Change the `main` method so that it looks like the one above to test the deletion operation.

You should see the following result:

```
Heap before deletion:
42 29 18 14 19 11 10
Heap after deletion:
29 19 18 14 10 11
```

When `deleteRoot` is called, the root node `42` gets deleted and replaced with the last element `10`. From there, heapify down occurs. `10` is smaller than its child so it gets swapped with the larger child `29`. `10` is now the left child of `29` and a parent to both `14` and `19`. However, as a parent, it is smaller than both its children so it gets swapped with the larger child `19`. After that, the heap structure is maintained.

Student's Code File

Here's the complete code for your `HeapImplementation.cpp` file:

```
class HeapImplementation {
private:
    int* heap;
    int size;
    int capacity;

public:
    HeapImplementation(int capacity) {
        heap = new int[capacity];
```

```
        this->capacity = capacity;
        size = 0;
    }

    void resize() {
        int* resizedHeap = new int[capacity * 2];
        for (int i = 0; i < capacity; i++) {
            resizedHeap[i] = heap[i];
        }
        delete[] heap;
        heap = resizedHeap;
        capacity *= 2;
    }

    void insert(int value) {
        if (size == capacity) {
            resize();
        }
        heap[size] = value;
        heapifyUp();
        size++;
    }

    void heapifyUp() {
        int index = size;
        while (hasParent(index) && parent(index) < heap[index])
        {
            swap(getParentIndex(index), index);
            index = getParentIndex(index);
        }
    }

    bool hasParent(int i) { return getParentIndex(i) >= 0; }
    int parent(int i) { return heap[getParentIndex(i)]; }
    int getParentIndex(int i) { return (i - 1) / 2; }

    void swap(int i, int j) {
        int temp = heap[i];
        heap[i] = heap[j];
        heap[j] = temp;
    }

    int deleteRoot() {
        if (size == 0) {
            throw std::logic_error("Heap is empty");
        }

        int root = heap[0];
        heap[0] = heap[size - 1];
        size--;

        heapifyDown();

        return root;
    }

    void heapifyDown() {
        int index = 0;
```

```
        while (hasLeftChild(index)) {
            int largerChildIndex = getLeftChildIndex(index);
            if (hasRightChild(index) && rightChild(index) >
        leftChild(index)) {
                largerChildIndex = getRightChildIndex(index);
            }

            if (heap[index] < heap[largerChildIndex]) {
                swap(index, largerChildIndex);
            } else {
                break;
            }

            index = largerChildIndex;
        }
    }

    bool hasLeftChild(int i) { return getLeftChildIndex(i) <
        size; }
    bool hasRightChild(int i) { return getRightChildIndex(i) <
        size; }
    int leftChild(int i) { return heap[getLeftChildIndex(i)]; }
    int rightChild(int i) { return heap[getRightChildIndex(i)];
        }
    int getLeftChildIndex(int i) { return 2 * i + 1; }
    int getRightChildIndex(int i) { return 2 * i + 2; }

    //search method

    void printHeap() {
        for (int i = 0; i < size; i++) {
            std::cout << heap[i] << " ";
        }
        std::cout << std::endl;
    }
};

int main() {
    HeapImplementation heap(10);
    heap.insert(42);
    heap.insert(29);
    heap.insert(18);
    heap.insert(14);
    heap.insert(19);
    heap.insert(11);
    heap.insert(10);

    std::cout << "Heap before deletion:" << std::endl;
    heap.printHeap();

    heap.deleteRoot();

    std::cout << "Heap after deletion:" << std::endl;
    heap.printHeap();

    return 0;
}
```

The code should produce the initial heap: `42 29 18 14 19 11 10`. After deletion and heapifying down, you should see the following heap: `29 19 18 14 10 11`. This process maintains the max-heap property.

Searching a Heap

While heaps excel in managing elements based on priority, searching for arbitrary elements in a heap is not as efficient. This lesson explains how to search in a heap and discusses its limitations.

Understanding Heap Structure

A heap is a complete binary tree, typically represented as an array. The heap property ensures a parent-child relationship but does not maintain a fully sorted order, which makes searching for arbitrary elements less efficient compared to other data structures like AVL trees or red-black trees.

Implementing Search in a Heap

To search for an element in a heap, you need to check each element until you find the target. This process requires a linear search through the heap array.

Search Method Implementation

Here's how you can implement a basic search method in a heap:

```
bool search(int value) {
    // Implements a linear search through the heap
    // to find a specific value
    for (int i = 0; i < size; i++) {
        if (heap[i] == value) {
            return true;
        }
    }
    return false;
}
```

Add the above method to the `HeapImplementation` class.

Testing the Search method

Finally, let's test our `search` method with a revised `main` method in our `HeapImplementation` class.

```
int main() {
    HeapImplementation heap(10);
    heap.insert(15);
    heap.insert(10);
    heap.insert(20);

    int valueToSearch = 10;
    bool isFound = heap.search(valueToSearch);
    std::cout << "Is value " << valueToSearch << " in the heap?
        " << (isFound ? "true" : "false") << std::endl;

    return 0;
}
```

Change the `main` method so that it looks like the one above to test the search operation.

If successful, the code will produce: `Is value 10 in the heap? true`. The `search` method will look through the heap to determine if a particular element is found. In our case, `10` is in the heap so the code will print `true`.

Inefficiency of Searching in Heaps

While it's possible to search in a heap, the operation is inefficient ($O(n)$ time complexity) compared to balanced binary search trees or hash tables. In heaps, elements are not fully sorted, so you can't take advantage of binary search techniques.

Recommended Data Structures for Efficient Searching

For use cases where efficient searching is crucial, consider using data structures like:

- **AVL Trees**: Self-balancing binary search trees that maintain a strict balance, leading to efficient search operations.
- **Red-Black Trees**: Another type of self-balancing binary search tree, offering good worst-case guarantees for insertion, deletion, and searching.

Student's Complete Code File

Here is the complete code for your `HeapImplementation.cpp` file:

```
class HeapImplementation {
private:
    int* heap;
    int size;
    int capacity;

public:
```

```
HeapImplementation(int capacity) {
    heap = new int[capacity];
    this->capacity = capacity;
    size = 0;
}

void resize() {
    int* resizedHeap = new int[capacity * 2];
    for (int i = 0; i < capacity; i++) {
        resizedHeap[i] = heap[i];
    }
    delete[] heap;
    heap = resizedHeap;
    capacity *= 2;
}

void insert(int value) {
    if (size == capacity) {
        resize();
    }
    heap[size] = value;
    heapifyUp();
    size++;
}

void heapifyUp() {
    int index = size;
    while (hasParent(index) && parent(index) < heap[index])
    {
        swap(getParentIndex(index), index);
        index = getParentIndex(index);
    }
}

bool hasParent(int i) { return getParentIndex(i) >= 0; }
int parent(int i) { return heap[getParentIndex(i)]; }
int getParentIndex(int i) { return (i - 1) / 2; }

void swap(int i, int j) {
    int temp = heap[i];
    heap[i] = heap[j];
    heap[j] = temp;
}

int deleteRoot() {
    if (size == 0) {
        throw std::logic_error("Heap is empty");
    }

    int root = heap[0];
    heap[0] = heap[size - 1];
    size--;

    heapifyDown();

    return root;
}
```

```
    void heapifyDown() {
        int index = 0;
        while (hasLeftChild(index)) {
            int largerChildIndex = getLeftChildIndex(index);
            if (hasRightChild(index) && rightChild(index) >
        leftChild(index)) {
                largerChildIndex = getRightChildIndex(index);
            }

            if (heap[index] < heap[largerChildIndex]) {
                swap(index, largerChildIndex);
            } else {
                break;
            }

            index = largerChildIndex;
        }
    }

    bool hasLeftChild(int i) { return getLeftChildIndex(i) <
        size; }
    bool hasRightChild(int i) { return getRightChildIndex(i) <
        size; }
    int leftChild(int i) { return heap[getLeftChildIndex(i)]; }
    int rightChild(int i) { return heap[getRightChildIndex(i)];
        }
    int getLeftChildIndex(int i) { return 2 * i + 1; }
    int getRightChildIndex(int i) { return 2 * i + 2; }

    bool search(int value) {
        for (int i = 0; i < size; i++) {
            if (heap[i] == value) {
                return true;
            }
        }
        return false;
    }

    void printHeap() {
        for (int i = 0; i < size; i++) {
            std::cout << heap[i] << " ";
        }
        std::cout << std::endl;
    }
};

int main() {
    HeapImplementation heap(10);
    heap.insert(15);
    heap.insert(10);
    heap.insert(20);

    int valueToSearch = 10;
    bool isFound = heap.search(valueToSearch);
    std::cout << "Is value " << valueToSearch << " in the heap?
        " << (isFound ? "true" : "false") << std::endl;

    return 0;
}
```

You should see the following output:

```
Is value 10 in the heap? true
```

Heap Applications

Heaps, as specialized tree-based data structures, are instrumental in various computational tasks due to their efficient management of ordered data. This lesson explores key applications of heaps, highlighting their versatility and effectiveness in different scenarios.

Priority Queues

Priority queues are a significant application of heaps. They allow for the management of data where each element has a priority, and elements with higher priority are served before those with lower priority.

- **Max-Heap for Highest Priority**: In a max-heap, the highest priority element is always at the root, making it ideal for scenarios where quick access to the highest priority element is required.
- **Min-Heap for Lowest Priority**: Conversely, a min-heap serves the lowest priority element first.

Example of a Priority Queue using a Max-Heap:

```
      50
     /  \
   30    20
  / \   / \
 10 15 5   8
```

Heap Sort

Heap sort is an efficient sorting algorithm that leverages the properties of heaps. It organizes elements in a specific order, either ascending or descending, by building a heap from the input data and then extracting the elements from the heap to form a sorted array.

- **Process**: Build a max-heap from the unsorted array; then repeatedly remove the root (maximum element) and rebuild the heap until all elements are sorted.

Kth Largest Element in an Array

Heaps are particularly useful for finding the Kth largest (or smallest) element in an array. This is achieved by maintaining a min-heap (for the Kth largest element) or a max-heap (for the Kth smallest element) of size K.

- **Method**: Insert the first K elements into the heap, then for each new

element, if it's larger (or smaller) than the root, replace the root with this element and heapify.

Median Finding

Heaps can be used to efficiently find the median of a running stream of data. This involves using two heaps: a max-heap to store the smaller half of the numbers and a min-heap for the larger half.

- **Balancing**: Ensure both heaps are approximately equal in size. The median is either the maximum of the max-heap or the minimum of the min-heap, or the average of both if they have the same size.

Graph Algorithms

In graph algorithms like Dijkstra's shortest path and Prim's minimum spanning tree, heaps (often implemented as priority queues) are used to efficiently select the next node to process.

- **Dijkstra's Algorithm**: Uses a min-heap to find the node with the smallest distance from the source.
- **Prim's Algorithm**: Uses a min-heap to select the edge with the minimum weight that connects a new node to the growing spanning tree.

These applications demonstrate the flexibility and efficiency of heaps in various computational contexts, from sorting and selection problems to graph algorithms and real-time data processing.

Formative Assessment 1

Formative Assessment 2